

Prompt Ladder — Week 3

Shahzaib Pervez · AI/ML Intern, FlyRank AI

Placeholder data note: metrics below use illustrative numbers consistent with the 3x claim (NDCG@10: heuristic 0.31, decision tree 0.52, learned model 0.94, std dev ≈ 0.02 across 5 folds) since real Week 1 numbers aren't locked yet. Swap in actual results from RESULTS.md before posting.

Model	NDCG@10	Notes
Heuristic baseline (tuned)	0.31	recency + category weighting
Decision tree baseline (optimized)	0.52	depth-6, grid-searched
Learned ranking model	0.94	5-fold CV, std dev ≈ 0.02

Run 0 — Baseline (the embarrassing one)

PROMPT

Explain this data. (table pasted in)

OUTPUT

This table shows three models. The heuristic baseline scored 0.31, the decision tree scored 0.52, and the learned ranking model scored 0.94 on NDCG@10. The learned model performed the best, followed by the decision tree, and the heuristic baseline had the lowest score. This suggests that the learned model is more effective at ranking than the other two approaches.

Verdict: Could describe literally any three-row comparison table. Restates numbers, adds nothing.

Version 1 — Layer added: clearer goal

PROMPT

Explain this data. My goal is to understand why the learned model outperformed the baselines and whether that improvement is trustworthy enough to put in my portfolio.

OUTPUT

The learned model's 0.94 vs. the heuristic's 0.31 is roughly a 3x gain. Because the result is reported with a std dev of ≈ 0.02 across 5 folds rather than a single run, the gain is unlikely to be a fluke of one split — it holds up across resampling. The decision tree, at 0.52, shows some of the gain comes from moving beyond hand-written rules generally, but the learned model still captures meaningfully more signal than a tree can.

What changed in the prompt: Added an explicit goal (“understand why,” “trustworthy enough”).

What actually improved in the output: Stopped just restating rows and started reasoning about validity — it noticed and used the std dev, which the baseline output ignored entirely.

What still failed: Generic tone, no sense of who's reading this. Words like “meaningfully” and “roughly” are filler standing in for precision.

What I'd try next: Give it a reader, so it stops writing for nobody in particular.

Version 2 — Layer added: defined audience

PROMPT

(same as V1) + Write for an ML engineering hiring manager who will scrutinize methodology, not a general reader.

OUTPUT

A 3x NDCG@10 gain (0.31 → 0.94) only means something if the baseline wasn't a strawman. The heuristic here was tuned, not a placeholder rule, and the decision tree — also optimized via grid search — landed at 0.52, meaning a good chunk of the lift comes from moving past rules at all, not just from throwing a bigger model at the problem. The remaining gap between tree and learned model (0.52 → 0.94) is where the real signal-capture advantage shows up. Cross-validated std dev of 0.02 suggests this isn't a lucky split.

What changed in the prompt: Named a specific, skeptical reader instead of “explain this.”

What actually improved in the output: Stopped defining NDCG or explaining what a baseline is, and pre-empted the exact objection a hiring manager would raise — “was your baseline weak?” — without being told to do that explicitly.

What still failed: Still a wall of prose. A hiring manager skimming for 15 seconds won't extract the key fact fast enough.

What I'd try next: Force a scannable structure.

Version 3 — Layer added: specified output format

PROMPT

(same as V2) + Format the answer as three labeled sections: Result, Why It's Trustworthy, Open Risk. Keep the total under 120 words.

OUTPUT

Result: Learned ranking model scores 0.94 NDCG@10 vs. 0.31 for a tuned heuristic and 0.52 for an optimized decision tree — roughly a 3x gain over the heuristic.

Why It's Trustworthy: Both baselines were tuned, not strawmen, so the gain isn't inflated by an easy comparison. Std dev of 0.02 across 5 folds indicates the result holds across resampling.

Open Risk: Gain is reported on one dataset; unclear if it generalizes to a different query distribution.

What changed in the prompt: Named three sections and a word cap.

What actually improved in the output: Scannable in under 10 seconds — the exact failure mode of Run 0 is now structurally impossible to repeat.

What still failed: Still leans on soft words (“roughly,” “indicates”) instead of committing to precise claims.

What I'd try next: Add hard constraints against filler language.

Version 4 — Layer added: constraints

PROMPT

(same as V3) + No adjectives like “significant,” “robust,” or “cutting-edge.” Every claim must cite a specific number from the table. One sentence per section maximum.

OUTPUT

Result: Learned model: 0.94 NDCG@10; heuristic baseline: 0.31; decision tree baseline: 0.52 — a 3x gain over the heuristic.

Why It's Trustworthy: Both baselines were tuned (heuristic hand-weighted, tree grid-searched to depth 6), and the 0.94 held with a std dev of 0.02 across 5 folds.

Open Risk: All three numbers come from one dataset, so generalization to a different query distribution is untested.

What changed in the prompt: Banned filler adjectives, forced number-grounding, capped sentence count.

What actually improved in the output: Every remaining sentence is now falsifiable — a reader could check each claim against the table directly. Nothing left to hand-wave.

What still failed: Nothing obvious yet — this is the tightest version so far.

What I'd try next: Try adding a verification/self-check layer to see if it can catch its own unsupported claims.

Version 5 — Layer added: verification requirement

PROMPT

(same as V4) + Before finalizing, flag any claim you make that isn't directly supported by the numbers given, and state the assumption behind it if it isn't.

OUTPUT

Result: Learned model: 0.94 NDCG@10; heuristic baseline: 0.31; decision tree baseline: 0.52 — a 3x gain over the heuristic. (Note: “3x” is an approximation of $0.94/0.31 = 3.03x$, treated as exact.)

Why It's Trustworthy: Both baselines were tuned, and the 0.94 held with a std dev of 0.02 across 5 folds. (Assumption: “tuned” implies reasonable effort on both baselines — not independently verifiable from the table alone.)

Open Risk: All three numbers come from one dataset. (Flag: assuming “the numbers given” means this is the only evaluation run — no info on other datasets tested.)

■ THIS LAYER MADE IT WORSE

What changed in the prompt: Added a self-verification instruction on top of the V4 constraints.

What actually improved in the output: Nothing meaningfully — the core claims were already fully number-grounded by V4.

What still failed / got worse: Broke the “one sentence per section” constraint it was previously obeying, reintroduced hedging phrases, and the parenthetical caveats undercut the confident, scannable tone V4 had earned.

What I'd try next: Only invoke verification narrowly — “flag it only if NOT traceable to a specific number” — and explicitly restate the word/sentence caps so the new layer can't override earlier ones.

Final Prompt (cleaned up, reusable by a stranger)

You are explaining a model comparison table to an ML engineering hiring manager who will scrutinize methodology, not a general reader -- do not explain basic terms they'd already know.

Data:

[PASTE TABLE -- model names, metric, notes columns]

Write exactly three labeled sections:

- Result: state the top-line comparison, citing exact numbers from the table.
- Why It's Trustworthy: address whether the baselines were fairly tuned and whether the result is likely to hold up (e.g. variance across runs/folds), citing specific numbers.
- Open Risk: name the most legitimate reason a skeptical reader could still doubt this result.

Rules:

- One sentence per section, maximum.
- No adjectives like "significant," "robust," "cutting-edge," or similar filler.
- Every claim must cite a specific number from the table.
- Only flag an assumption if a claim in your answer is NOT directly traceable to a number in the table -- do not add general disclaimers otherwise.

Why this version is the reusable one: it carries goal, audience, format, and constraints — the layers that stacked cleanly — and narrows the verification instinct from V5 into something that only fires when actually needed, instead of adding hedging by default. Someone else on the track could drop in their own comparison table and get a tight, skeptic-proof summary without needing you in the room to explain what “good” looks like.